JAVA DSA CHEAT SHEET — Quick Reference Guide for LeetCode & Technical Interviews

```java
// Primitives & Arrays
int[] arr = new int[n];
int[][] mat = new int[r][c];

// Strings & Builders
String s = "hello";
StringBuilder sb = new StringBuilder();

// Collections
List<Integer> list = new ArrayList<>();
Set<Integer> set = new HashSet<>();
NavigableSet<Integer> ts = new TreeSet<>();
Map<K, V> map = new HashMap<>();
NavigableMap<K, V> tm = new TreeMap<>();
Map<K, V> lru = new LinkedHashMap<>(cap, 0.75f, true);

// Stacks, Queues & Heaps
Queue<Integer> q = new LinkedList<>();
Deque<Integer> st = new ArrayDeque<>();
PriorityQueue<Integer> minH = new PriorityQueue<>();
PriorityQueue<Integer> maxH =
    new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> heap =
    new PriorityQueue<>((a, b) -> Integer.compare(a[0], b
        [0]));
```

Listing 1: 1. DECLARATIONS INITIALIZATION

```java
// String (Immutable)
s.length();              s.charAt(i);            s.
    toCharArray();
s.substring(beg, end);   s.equals(other);        s.
    equalsIgnoreCase(other);
s.compareTo(other);      s.indexOf(sub);         s.
    lastIndexOf(sub);
s.startsWith(prefix);    s.endsWith(suffix);     s.
    contains(seq);
s.toLowerCase();         s.toUpperCase();        s.trim
    ();
s.replace(old, new);     s.replaceAll(rgx, rep); s.split
    (regex);
String.join(delim, elems);

// StringBuilder (Mutable)
sb.append(val);          sb.insert(off, val);    sb.
    delete(start, end);
sb.deleteCharAt(idx);    sb.replace(s, e, str);  sb.
    reverse();
sb.setCharAt(idx, ch);   sb.toString();
```

Listing 2: 2. STRING STRINGBUILDER

```java
Arrays.sort(arr);                          // O(N log N)
Arrays.sort(arr, comparator);              // Custom sort
Arrays.parallelSort(arr);                  // Multi-
    threaded
Arrays.binarySearch(arr, key);             // Pre-sorted O(
    log N)
Arrays.fill(arr, val);                     // Fill all
Arrays.fill(arr, from, to, val);           // Fill range
Arrays.copyOf(orig, newLength);            // Resize/copy
Arrays.copyOfRange(orig, from, to);        // Range copy
Arrays.asList(T... a);                     // Fixed-size
    List
Arrays.toString(arr);                      // 1D String
Arrays.deepToString(mat);                  // 2D String
Arrays.equals(a1, a2);                     // 1D equality
Arrays.deepEquals(m1, m2);                 // 2D equality
```

Listing 3: 3. ARRAYS UTILITY CLASS

```java
list.add(e);             list.add(idx, e);       list.
    addAll(coll);
list.get(idx);           list.set(idx, element);
list.remove(idx);        list.remove(Object o);  list.
    removeIf(pred);
list.size();             list.isEmpty();         list.
    clear();
list.contains(o);        list.indexOf(o);        list.
    lastIndexOf(o);
list.subList(from, to); // View range             list.
    sort(comparator);
```

Listing 4: 4. LIST / ARRAYLIST

```java
// HashSet - O(1)
set.add(e);              set.remove(o);          set.
    contains(o);
set.size();              set.isEmpty();          set.
    clear();
set.addAll(c);           // Union
set.retainAll(c);        // Intersection
set.removeAll(c);        // Difference

// TreeSet / NavigableSet - O(log N)
treeSet.first();         treeSet.last();         // Min
    / Max
treeSet.pollFirst();     treeSet.pollLast();     //
    Extract Min/Max
treeSet.higher(e);       // Strictly > e
treeSet.lower(e);        // Strictly < e
treeSet.ceiling(e);      // Smallest >= e
treeSet.floor(e);        // Largest <= e
```

Listing 5: 5. SET (HASHSET TREESET)

```java
// HashMap - O(1)
map.put(k, v);           map.putIfAbsent(k, v);
map.get(k);              map.getOrDefault(k, defaultVal);
map.remove(k);           map.remove(k, v);
map.containsKey(k);      map.containsValue(v);
map.keySet();            map.values();           map.
    entrySet();
map.computeIfAbsent(k, key -> new ArrayList<>());
map.computeIfPresent(k, (key, val) -> val + 1);
map.merge(k, v, (oldV, newV) -> oldV + newV);

// TreeMap / NavigableMap - O(log N)
treeMap.firstKey();      treeMap.lastKey();      // Min
    / Max key
treeMap.pollFirstEntry();treeMap.pollLastEntry(); // Poll
     entry
treeMap.higherKey(k);    treeMap.lowerKey(k);    // > /
    < k
treeMap.ceilingKey(k);   treeMap.floorKey(k);   // >= /
    <= k
treeMap.subMap(fromKey, toKey);                  // Sub-
    map view
```

Listing 6: 6. MAP (HASHMAP TREEMAP)

```java
// Queue Interface (FIFO)
q.offer(e);              // Enqueue (safe)
q.poll();                // Dequeue (null if empty)
q.peek();                // Front element

// Stack via ArrayDeque (LIFO)
st.push(e);              // Push top
st.pop();                // Pop top
st.peek();               // View top

// Double-Ended Queue (Deque)
dq.addFirst(e);          dq.offerFirst(e);       dq.
    peekFirst();
dq.addLast(e);           dq.offerLast(e);        dq.
    peekLast();
dq.pollFirst();          dq.pollLast();
```

Listing 7: 7. QUEUE, DEQUE STACK

```java
pq.offer(e);             // Insert O(log N)
pq.poll();               // Extract root O(log N)
pq.peek();               // View root O(1)
pq.remove(obj);          // Remove target O(N)
pq.size();               pq.isEmpty();           pq.
    clear();
```

Listing 8: 8. PRIORITYQUEUE (HEAPS)

```java
Collections.sort(list);                  Collections.sort
    (list, comp);
Collections.reverse(list);               // In-place
    reverse
Collections.swap(list, i, j);            // Swap elements
Collections.max(coll);                   Collections.min(
    coll);
Collections.frequency(coll, obj);        // Frequency
    count
Collections.binarySearch(list, key);     // Binary search
Collections.reverseOrder();              // Reverse
    comparator
```

Listing 9: 9. COLLECTIONS UTILITY CLASS

```java
// Character Helpers
Character.isLetterOrDigit(ch); Character.isDigit(ch);
    Character.isLetter(ch);
Character.toLowerCase(ch);     Character.toUpperCase(ch);

// Parsing, Math & Bitwise
Integer.parseInt(str);         String.valueOf(val);
Integer.bitCount(n);           // Count 1-bits
Integer.highestOneBit(n);      // MSB
Math.max(a, b);                Math.min(a, b);
    Math.abs(x);
Math.pow(base, exp);           Math.sqrt(x);
```

Listing 10: 10. DSA HELPER UTILITIES